

Ejercicio 1

a) normalize

Elimina partes innecesarias de una ruta como .. o .

Ejemplo:

```
import java.nio.file.Path;
import java.nio.file.Paths;

public class EjemploNormalize {

    public static void main(String[] args) {

        Path ruta = Paths.get("C:/temp/./documentos/archivo.txt")

        Path normalizada = ruta.normalize();

        System.out.println(normalizada);

    }

}
```

b) resolve

Une rutas.

Ejemplo:

```
import java.nio.file.Path;
import java.nio.file.Paths;

public class EjemploResolve {

    public static void main(String[] args) {

        Path carpeta = Paths.get("C:/temp");

        Path archivo = carpeta.resolve("datos.txt");

        System.out.println(archivo);

    }

}
```

c) relativize

Obtiene la ruta relativa entre dos rutas.

Ejemplo:

```
import java.nio.file.Path;
import java.nio.file.Paths;

public class EjemploRelativize {

    public static void main(String[] args) {

        Path ruta1 = Paths.get("C:/temp");

        Path ruta2 = Paths.get("C:/temp/documentos/foto.jpg");

        Path relativa = ruta1.relativize(ruta2);

        System.out.println(relativa);

    }

}
```

Ejercicio 2

```
import java.nio.file.Path;
import java.nio.file.Paths;

public class Ejercicio2 {

    public static void main(String[] args) {

        Path ruta = Paths.get("C:/temp/aulas/ada/");

        Path subruta = ruta.subpath(1, 3);

        System.out.println(subruta);

    }

}
```

Ejercicio 3

```
import java.io.IOException;
import java.nio.file.*;

public class Ejercicio3 {

    public static void main(String[] args) {

        Path carpeta = Paths.get("C:/temp");

        try {

            DirectoryStream<Path> archivos =
                Files.newDirectoryStream(carpeta);

            for (Path archivo : archivos) {

                if (Files.isRegularFile(archivo)) {

                    String nombre = archivo.getFileName().toString();

                    int punto = nombre.lastIndexOf(".");

                    if (punto != -1) {

                        String extension =
                            nombre.substring(punto + 1);

                        System.out.println(
                            nombre + " -> " + extension);

                    }

                }

            }

        } catch (IOException e) {

            System.out.println(e.getMessage());

        }

    }

}
```

Ejercicio 4

```
import java.io.IOException;
import java.nio.file.*;
import java.util.stream.Stream;
public class Ejercicio4 {
    public static void main(String[] args) {
        Path ruta = Paths.get("C:/temp");
        int archivos = 0;
        int directorios = 0;
        try (Stream<Path> paths = Files.walk(ruta)) {
            for (Path p : paths.toList()) {
                if (Files.isRegularFile(p)) {
                    archivos++;
                }
                if (Files.isDirectory(p)) {
                    directorios++;
                }
            }
            System.out.println("Archivos: " + archivos);
            System.out.println("Directorios: " + directorios);
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Ejercicio 5

```
import java.io.IOException;
import java.nio.file.*;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class Ejercicio5 {

    public static List<Path> buscarExtension(
        String directorio,
        String extension) {
        List<Path> lista = null;
        try (Stream<Path> paths =
            Files.walk(Paths.get(directorio))) {
            lista = paths
                .filter(Files::isRegularFile)
                .filter(p -> p.toString()
                    .endsWith(extension))
                .collect(Collectors.toList());
            lista.forEach(System.out::println);
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
        return lista;
    }

    public static void main(String[] args) {
        buscarExtension("C:/temp", ".txt");
    }
}
```

Ejercicio 6

```
import java.io.IOException;
import java.nio.file.*;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class Ejercicio6 {

    public static List<Path> buscarArchivosGrandes(
        String directorio) {
        List<Path> lista = null;
        try (Stream<Path> paths =
            Files.walk(Paths.get(directorio))) {
            lista = paths
                .filter(Files::isRegularFile)
                .filter(p -> {
                    try {
                        return Files.size(p)
                            >= 5 * 1024 * 1024;
                    } catch (IOException e) {
                        return false;
                    }
                })
                .collect(Collectors.toList());
            lista.forEach(System.out::println);
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
        return lista;
    }
}
```

```
public static void main(String[] args) {  
    buscarArchivosGrandes("C:/temp");  
}  
}
```